

Teaching Notes — Day 4 & Day 5

- Semantic HTML, Tables, Accessibility & Mini Project (College Student Portal)
- Session Time Plan
- SEGMENT 1: Introduction & Recap (5 min)
- SEGMENT 2: Semantic HTML Elements (35 min)
 - 2.1 Why Semantic HTML Matters (5 min)
 - 2.2 The Semantic Elements — One by One (25 min)
 - 2.3 Putting It All Together — Full Skeleton (5 min)
- SEGMENT 3: Tables (25 min)
 - 3.1 When to Use Tables (3 min)
 - 3.2 Table Structure (12 min)
 - 3.3 colspan and rowspan (10 min)
- SEGMENT 4: Accessibility Basics (20 min)
 - 4.1 Alt Text (5 min)
 - 4.2 Labels for Form Fields (5 min)
 - 4.3 Semantic Structure Recap (5 min)
 - 4.4 Keyboard Accessibility (5 min)
- || 5-Minute Break
- SEGMENT 5 (Day 5 start): Mini Project 1 — College Student Portal (30 min)
 - 5.1 Project Brief (5 min)
 - 5.2 Planning the Site Structure (10 min)
 - 5.3 Page-by-Page Content Plan (10 min)
 - 5.4 Assign the Task
- SEGMENT 6: Wrap-up, Q&A, Homework (5-10 min)
 - Recap Checklist (read aloud, have students self-check)
 - Homework
 - Quick Quiz Ideas (optional, for next class opener)
 - Instructor Notes / Tips

Teaching Notes — Day 4 & Day 5

Semantic HTML, Tables, Accessibility & Mini Project (College Student Portal)

Total Duration: 2 hours (120 minutes) **Audience:** Beginner web development students (already familiar with basic HTML tags, divs, headings, links)

Session Time Plan

Segment	Topic	Time
1	Introduction & Recap	5 min
2	Semantic HTML Elements	35 min
3	Tables	25 min
4	Accessibility Basics	20 min
5	Break	5 min
6	Day 5 — Mini Project Kickoff: College Student Portal	30 min
7	Wrap-up, Q&A, Homework	5-10 min

SEGMENT 1: Introduction & Recap (5 min)

Talking points: - Quick recap: so far students have learned basic tags — `<div>`, `<span>`, `<h1>`–`<h6>`, `<p>`, `<a>`, `<img>`, lists. - Ask: “If I gave you a raw HTML file using only `<div>` tags, could you tell where the navigation, main content, and footer are just by looking at the tag names?” - Answer: No — this is the problem **semantic HTML** solves.

Write on board: > “Semantic HTML = using tags that describe their *meaning*, not just their *appearance*.”

SEGMENT 2: Semantic HTML Elements (35 min)

2.1 Why Semantic HTML Matters (5 min)

Explain three concrete benefits — write these as the “3 pillars”:

1. **Readability for developers** — code becomes self-documenting. A new developer reading `<nav>` instantly knows what it does; `<div class="nav">` requires trusting the class name is accurate and unchanged.
2. **Accessibility** — screen readers use semantic tags to build a “landmarks” list, letting visually impaired users jump directly to navigation, main content, or footer instead of listening to the entire page.
3. **SEO (Search Engine Optimization)** — search engines weigh content inside `<article>` or `<main>` more heavily than generic `<div>` soup, improving how your page ranks and is summarized.

Analogy to use in class: > “A `<div>` is like an unlabeled cardboard box. A `<header>` or `<article>` is a labeled box — anyone (including a machine) instantly knows what’s inside without opening it.”

2.2 The Semantic Elements — One by One (25 min)

For each tag: explain purpose → show a code snippet → point out common mistakes.

`<header>`

- Represents introductory content for a page **or** a section (not just the very top of the site).
- Typically holds: logo, site title, tagline, sometimes primary navigation.
- **Note:** A page can have multiple `<header>` elements — one for the page, one inside an `<article>`.

```
<header>
  <h1>College Student Portal</h1>
  <p>Empowering students, one click at a time</p>
</header>
```

`<nav>`

- Wraps major navigation links (main menu, breadcrumbs, pagination, table of contents).

- **Common mistake:** wrapping *every* group of links in <nav> (e.g., footer social icons). Reserve <nav> for significant navigation blocks only.

```
<nav>
  <ul>
    <li><a href="index.html">Home</a></li>
    <li><a href="about.html">About</a></li>
    <li><a href="courses.html">Courses</a></li>
  </ul>
</nav>
```

<main>

- Wraps the dominant, unique content of the page — content that is NOT repeated across pages (so not headers, nav, sidebars, footers).
- **Rule:** Only one <main> per page, and it should not be nested inside <article>, <aside>, <header>, <footer>, or <nav>.

```
<main>
  <h2>Welcome to Our College</h2>
  <p>Explore our courses, faculty, and campus life...</p>
</main>
```

<section>

- A thematic grouping of content, generally with its own heading.
- Ask students: “Is this content a distinct ‘chapter’ of the page?” If yes → <section>.

```
<section>
  <h2>Our Departments</h2>
  <p>We offer programs in Computer Science, Commerce, and Arts.</p>
</section>
```

<article>

- Self-contained content that could be lifted out and make sense on its own (blog post, news story, faculty profile card, forum post).
- **Test to teach students:** “If I copy-pasted this block into an RSS feed or another website, would it still make complete sense?” If yes → <article>.

```
<article>
  <h3>Dr. Meera Sharma</h3>
  <p>Professor of Computer Science, specializing in AI and Machine Learning.</p>
</article>
```

<section> vs <article> — the #1 point of confusion. Spend 2 extra minutes here: - <article> = independent, reusable unit. - <section> = a grouped chunk *within* a page’s larger flow — not necessarily independent. - An <article> can contain <section>s (e.g., a blog post with “Introduction” and “Conclusion” sections). A <section> can contain multiple <article>s (e.g., a “Latest News” section listing several articles).

<aside>

- Content tangentially related to the main content — sidebars, pull quotes, ads, related links.

```
<aside>
  <h4>Quick Links</h4>
```

```

<ul>
  <li><a href="#">Exam Schedule</a></li>
  <li><a href="#">Library Portal</a></li>
</ul>
</aside>

```

<footer>

- Closing information for a page or section: copyright, contact info, social links, sitemap.
- Like <header>, can appear multiple times (page footer + article footer).

```

<footer>
  <p>&copy; 2026 XYZ College. All rights reserved.</p>
</footer>

```

<figure> and <figcaption>

- <figure> wraps self-contained media (image, diagram, code snippet, chart) that is referenced from the main flow but can be moved without affecting meaning.
- <figcaption> provides a caption — always the first or last child of <figure>.

```

<figure>
  
  <figcaption>Fig 1: Our campus spans 50 acres.</figcaption>
</figure>

```

2.3 Putting It All Together — Full Skeleton (5 min)

Draw this on the board / project it live:

```

<body>
  <header>...</header>
  <nav>...</nav>
  <main>
    <section>
      <article>...</article>
      <article>...</article>
    </section>
    <aside>...</aside>
  </main>
  <footer>...</footer>
</body>

```

Classroom activity (optional, 3 min): Show a page built entirely with <div>s and ask students to call out which semantic tag should replace each one.

SEGMENT 3: Tables (25 min)

3.1 When to Use Tables (3 min)

Golden rule to emphasize strongly: > “Tables are for **tabular data** (rows and columns of related data) — NEVER for page layout. That was a bad practice from the 1990s.”

Good use cases: timetables, marksheets, fee structures, comparison charts.

3.2 Table Structure (12 min)

Tag	Purpose
<table>	Root container for the whole table
<thead>	Groups header row(s)
<tbody>	Groups the main data rows
<tfoot>	Groups footer row(s) (totals, summaries) — mention briefly
<tr>	Table row
<th>	Header cell (bold, centered by default; use scope attribute for accessibility)
<td>	Standard data cell

```
<table>
  <thead>
    <tr>
      <th>Course</th>
      <th>Duration</th>
      <th>Fees</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>B.Sc Computer Science</td>
      <td>3 Years</td>
      <td>₹90,000</td>
    </tr>
    <tr>
      <td>B.Com</td>
      <td>3 Years</td>
      <td>₹70,000</td>
    </tr>
  </tbody>
</table>
```

Walk through this line by line, explaining why <thead>/<tbody> separation matters (styling, accessibility, and allows browsers to keep header visible while scrolling body in some UIs).

3.3 colspan and rowspan (10 min)

- **colspan**: merges a cell across multiple *columns*.
- **rowspan**: merges a cell across multiple *rows*.

Example — class timetable with a merged lunch break:

```
<table border="1">
  <thead>
    <tr>
      <th>Time</th>
      <th>Monday</th>
      <th>Tuesday</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>9:00 - 10:00</td>
      <td>Maths</td>
      <td>Physics</td>
    </tr>
    <tr>
```

```

        <td>1:00 - 2:00</td>
        <td colspan="2">Lunch Break</td>
    </tr>
    <tr>
        <td rowspan="2">Lab Session</td>
        <td>Chemistry Lab</td>
        <td>Computer Lab</td>
    </tr>
</tbody>
</table>

```

Teaching tip: Draw a 3x3 grid on the whiteboard first, then physically cross out cells to show what “merging” looks like before writing the code — this visual step prevents most student confusion.

Common student mistake: Forgetting that when you use `colspan`, you must include *fewer* `<td>` tags in that row (since the merged cell counts for multiple columns). Emphasize counting columns carefully.

SEGMENT 4: Accessibility Basics (20 min)

Framing statement: > “Accessibility (a11y) means making your website usable by everyone — including people using screen readers, keyboard-only navigation, or with visual/motor impairments. It’s not optional polish; in many countries it’s a legal requirement.”

4.1 Alt Text (5 min)

- Every meaningful `<img>` needs a descriptive `alt` attribute.
- Decorative images (pure styling, no informational value) should use `alt=""` (empty, not omitted) so screen readers skip them.

```

<!-- Good -->


<!-- Decorative -->


```

Rule of thumb to teach: “If you removed the image, what one sentence would you need to add to the page so no information is lost? That’s your alt text.”

4.2 Labels for Form Fields (5 min)

- Every input needs an associated `<label>` — this helps screen readers announce the field’s purpose, and lets sighted users click the label to focus the input.

```

<label for="studentName">Full Name:</label>
<input type="text" id="studentName" name="studentName">

```

Point out: the `for` attribute value must exactly match the input’s `id`.

4.3 Semantic Structure Recap (5 min)

- Connect back to Segment 2: using `<nav>`, `<main>`, `<header>`, `<footer>` isn’t just good practice — it directly powers screen-reader “landmark navigation,” letting users jump between regions with a keystroke instead of tabbing through everything.

- Heading hierarchy matters too: don't skip from <h1> to <h4> — screen reader users navigate by heading levels like a table of contents.

4.4 Keyboard Accessibility (5 min)

- Every interactive element (links, buttons, form fields) must be reachable and operable using **only the Tab key and Enter/Space** — no mouse.
- Avoid using <div onclick="..."> for buttons — a <div> isn't focusable by default. Use <button> instead, which comes with keyboard support built in.
- Mention tabindex briefly: tabindex="0" adds an element to natural tab order; avoid positive values (they create confusing custom orders).

Quick class demo (2 min): Ask a student to try navigating a sample page using only Tab/Enter — no mouse — to feel the real experience.

II 5-Minute Break

SEGMENT 5 (Day 5 start): Mini Project 1 — College Student Portal (30 min)

5.1 Project Brief (5 min)

Present the goal: > “You will build a multi-page static website for a fictional college, applying everything from Day 4: semantic tags, tables, and accessibility.”

Required pages: 1. Home 2. About 3. Courses 4. Faculty 5. Students 6. Contact 7. Registration

5.2 Planning the Site Structure (10 min)

Recommended folder structure — write on board:

```
college-portal/
|
|— index.html           (Home)
|— about.html
|— courses.html
|— faculty.html
|— students.html
|— contact.html
|— registration.html
|
|— css/
|   |— style.css
|— images/
|   |— (logo, photos, etc.)
```

Discuss: every page shares the same <header>, <nav>, and <footer> — this is where semantic HTML consistency pays off across a real multi-page project.

5.3 Page-by-Page Content Plan (10 min)

Go through each page and specify which tags apply — this becomes their working checklist:

Page	Key Content	Semantic Tags to Use
Home	Welcome banner, highlights, quick links	<header>, <nav>, <main>, <section> (highlights), <aside> (quick links), <footer>
About	College history, mission, vision	<main>, <section> for each (History / Mission / Vision), <figure> for campus image
Courses	List/table of courses with duration & fees	<main>, <table> with <thead>/<tbody>, possible colspan for combined fee rows
Faculty	Faculty profile cards	<main>, <section> "Our Faculty", each profile as <article>
Students	Student achievements / testimonials	<main>, <section>, <article> per testimonial, <figure> for photos
Contact	Address, map, contact form	<main>, <address> tag (mention this new tag), form with proper <label>s
Registration	Registration form	<main>, form with <label>, fieldset/legend (mention briefly), keyboard-accessible inputs

Mini live-code demo (if time allows): Build the Home page skeleton together as a class, narrating each semantic choice out loud.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Home - College Student Portal</title>
</head>
<body>
  <header>
    <h1>XYZ College Student Portal</h1>
  </header>

  <nav>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="about.html">About</a></li>
      <li><a href="courses.html">Courses</a></li>
      <li><a href="faculty.html">Faculty</a></li>
      <li><a href="students.html">Students</a></li>
      <li><a href="contact.html">Contact</a></li>
      <li><a href="registration.html">Registration</a></li>
    </ul>
```

```

</nav>

<main>
  <section>
    <h2>Welcome to XYZ College</h2>
    <p>Shaping futures since 1990.</p>
  </section>

  <section>
    <h2>Why Choose Us?</h2>
    <article>
      <h3>Expert Faculty</h3>
      <p>Learn from experienced professors across every
department.</p>
    </article>
    <article>
      <h3>Modern Campus</h3>
      <p>State-of-the-art labs, library, and sports facilities.
</p>
    </article>
  </section>

  <aside>
    <h4>Quick Links</h4>
    <ul>
      <li><a href="registration.html">Apply Now</a></li>
      <li><a href="courses.html">View Courses</a></li>
    </ul>
  </aside>
</main>

<footer>
  <p>&copy; 2026 XYZ College. All rights reserved.</p>
</footer>
</body>
</html>

```

5.4 Assign the Task

Break students into pairs or individual work: - Everyone builds index.html (Home) together in the remaining class time (or as guided homework). - Remaining 6 pages assigned as homework for the next session, using the table in 5.3 as their spec.

SEGMENT 6: Wrap-up, Q&A, Homework (5-10 min)

Recap Checklist (read aloud, have students self-check)

- ☐ I can explain the difference between <section> and <article>.
- ☐ I know when to use <aside> vs <main>.
- ☐ I can build a table using thead/tbody/tr/th/td.
- ☐ I can merge table cells using colspan and rowspan.
- ☐ I always add meaningful alt text to images.
- ☐ I always link <label for> to an input's id.
- ☐ I understand why keyboard navigation matters.

Homework

1. Complete the remaining 6 pages of the College Student Portal (About, Courses, Faculty, Students, Contact, Registration).
2. Courses page **must** include a table with at least one colspan or rowspan example.
3. Registration page **must** include a form where every input has a properly linked <label>.
4. Test every page using only the Tab key — no mouse — and fix any element that can't be reached.

Quick Quiz Ideas (optional, for next class opener)

1. What's the difference between <section> and <article>? Give one example of each.
 2. Which tag would you use for a sidebar of "related courses"?
 3. Write the HTML to merge two table cells horizontally.
 4. Why should <div onclick="..."> be avoided in favor of <button>?
 5. What does alt="" (empty) signal to a screen reader, versus omitting alt entirely?
-

Instructor Notes / Tips

- Keep a live-coding editor open throughout (e.g., VS Code + Live Server) — students retain semantic tag purposes far better when they see instant rendering than from slides alone.
- For the <section> vs <article> confusion, revisit it again briefly during the project segment — it's the most common real-world mistake.
- If time is short, the Accessibility segment can be trimmed to 15 min by merging 4.1 (alt text) and 4.2 (labels) into a single "content accessibility" mini-topic, keeping keyboard accessibility as its own point (it's the one most often skipped by students).